

Slipshow: chill-coding with OCaml

Paul-Elliot

July 12, 2026

1 Introduction

Slipshow is a tool to make presentation. It tries to innovate from the traditional “deck of slides” that rules the current presentations. For instance, it introduces the concept of slips, that are slides without a bottom limit, on which you scroll to display the content. Another feature that is unique to Slipshow, as far as I know, is the ability to record a drawing, edit it, and replay it during the presentation.

2 Growing the project

Slipshow started as a single file containing some JavaScript, that could be included in an HTML page written in a specific way, to turn it into a presentation. However, this very quickly became unmaintainable. Basic features were buggy and endlessly fixed, breaking other part of the program.

It is now a relatively big project, that features a runtime, a compiler, an LSP server with a preview window, a record-and-replay for hand drawings with a specialized user interface to edit them. How did we go from a simpler thing being unmaintainable, to the current version?

2.1 Chill-coding

Today, the answer for such question is very often “vibe-coding”. Vibe-coding is the practice of relying entirely on AI for generating the code, without even reading it. Freed from the friction of interacting with the code, considered too low-level, the cognitive load is lowered. You can start building with your vibe.

At first, from the name “vibe-code” and the good vibes that I get while coding, I thought I was practicing it. The first issue is that I am not using AI to generate any code. The second issue is that vibe-coding implies losing the knowledge of the internals of your software, which can make it hard to really trust enough to distribute it, at least not without really extensive testing.

I like to know the internals of my softwares, and more than vibes, I feel *chill* while coding. Chill when doing big refactors. Chill when adding new features. Chill for feeling safe from introducing bugs or complexity to the codebase. Which is why I introduce the term *chill-coding*, for the act of coding without fear.

Chill-coding requires either exceptional skills, or support from powerful tools. I fall into the second category. After being stuck for years with the JavaScript

prototype, I moved to OCaml as the primary language for Slipshow. Relying on the excellent compiler errors, state-of-the-art editor support, exceptional library ecosystem and over-the-top community, I was able to *chill-code* all the way.

3 An experience report

I have been very happy with the experience of writing in OCaml since the beginning. As mentioned above, it feels safe and lets your creativity wander, without friction. My hope is to share a bit of this excitement for the language, the compiler and the tools around, as well as the OCaml ecosystem and community, through a selected (git) history of Slipshow.

3.1 A runtime in JavaScript

As mentioned, Slipshow started as a presentation engine written in JavaScript. Here is an excerpt of commits all focused on fixing the same basic feature: going back in the presentation:

```
3af56a3 — corrected horribug previous
436bb65 — implemented engine.previousSlip ← the feature is born
d0cfc69 — corrected "not coming beginning on previous" bug
61e6cb7 — fix "future slip, delay not 0 again when previousing"
3af56a3 — corrected horribug previous
bcd8323 — better previous: use delay from previous state
28372cd — two bugs in previous and annotations
2437301 — correcting bug with "previous" and "resizeObserver"
```

3.2 OCaml for the compiler

In 0fa489e, the first line of OCaml is introduced in the project. It is the commit introducing the compiler, that takes a markdown file and outputs a standalone HTML file.

OCaml is known to be very good for compiler implementation. It is no exception in this case, especially when relying on `cmdliner` for the command line, `grace` for error reporting, and `cmarkit` for Markdown parsing.

One relatively exotic feature from OCaml that I am benefiting from are extensible variants. `cmarkit` defines its AST using extensible variants, allowing the user of the library to extend it. Note however that with extensible variants, one loses some kind of chill points, as exhaustiveness checks cannot be made for extensible variants. In commit 5713489 could get the chill back with:

```
type s_block =
| Slip of Block.t attributed node
| SlipScript of Block.Code_block.t attributed node
| [...]
```

```
type Cmarkit.Block.t += S_block of s_block
```

```
5713489 — Make it more suitable for match exhaustiveness check.
```

3.3 The runtime, rewritten in OCaml

For some time, the JavaScript code for the presentation engine, and the OCaml code for the compiler, co-existed. However, I still had the issues mentioned above on maintainability, and making the two communicate was challenging. So it was time to rewrite it all in OCaml.

The best way to summarize the rewriting is the following commit:

32d0917 — “BOOOOOOOOOOOOOOOOOhMM!” (*Undo monad, they work*)

This commit is about the same feature that the commits mentioned in section 3.1 were trying to fix. But I’ve already spoke about the Undo monad enough, so in this section, I’ll focus on how `dune`, `js_of_ocaml` and `ppx_blob` made it absolutely trivial to drive the build of Slipshow, which is more complex than usual: The Slipshow runtime is compiled to bytecode, then to JavaScript, and finally embedded in the source of the Slipshow compiler.

```
(executable
  (name engine)
  (modes js))

(executable
  (name compiler)
  (preprocess (pps ppx_blob)) ; Use with [%blob "engine.bc.js"]
  (preprocessor_deps engine.bc.js))
```

3.4 Record and replay drawings

One recent and game-changing addition to Slipshow is the ability to record drawings, edit them and replay the result during the presentation.

The UI to edit records, make very much like a video editor, despite the expected challenge to craft, proved to be a joy to make. This joy can be summed up with the following commit:

3ac99e3 — “Wo so cool lwd”

Again, the joy comes from high quality libraries from the ecosystem. `Lwd` is a library for reactive programming. It allows to define the UI from reactive values, and have the UI update as the reactive values change. Here, we shorten path to what was written at `elapsed_time`:

```
let filter_path elapsed_time path =
  let$* path = path in (* the two inputs are reactive values *)
  let$ elapsed_time = elapsed_time in
  List.filter (fun (_, t) -> t < elapsed_time) path
```

4 Conclusion

If Slipshow was able to grow, if I am able to chill-code it and have fun all days, this is thanks to an incredible language, compiler, ecosystem and community. I hope to be able to send this message and share my excitement.